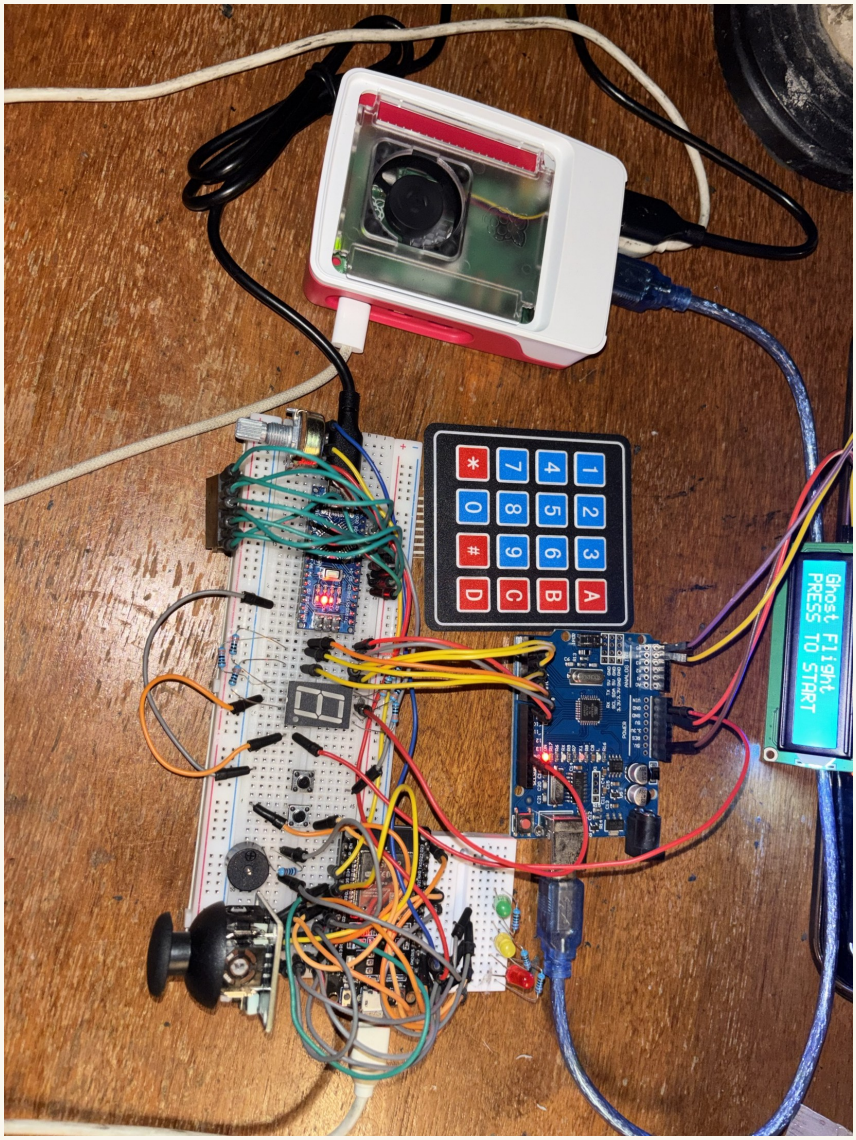
A PHYSICAL HALLOWEEN GAME / ADRIEL MAGALONA

Dead Channel

Please Hold.

Six callers. One night shift.
Do not wake the operator.



01 / Real prototype, photographed by the builder. The LCD shows the Ghost Flight menu; this shared console also runs Please Hold.

An illustrated engineering dossier for a haunted telephone exchange: real tuning, physical dialing, deliberate routing and a connection you must know when to release.

15 pages / 6 editable diagrams / 3 USB stations / pin-level netlist
Prototype documentation. Gameplay video will be added as a separate media link.

Every part changes the play

Six written callers ask to return to six places. The player recovers their signals, enters an extension, interprets a message and makes a dangerous connection. The hardware creates a sequence of decisions across the desk.

Component	Purpose in Please Hold
Potentiometer	Find and maintain the caller carrier; a stable signal reveals the extension.
16x2 LCD	Call information and extension entry. Eye-release warnings deliberately remain elsewhere.
4x4 keypad / Nano	Enter four digits, # to submit, * to clear. Correct authorization does not choose the destination.
Joystick / ESP32	Choose the destination from the caller clue. Click pauses.
Main button	Hold to connect; release preserves partial progress.
Green / yellow / red	Safe / warning / watched. Red means release.
Separate hang-up button	Reduce attention at the cost of extension, authorization and partial progress.
5161AS digit / Uno	Count the six remaining callers down to zero.
Buzzer + browser sound	Brief call, warning and result cues. Visual information remains sufficient.

The Halloween idea

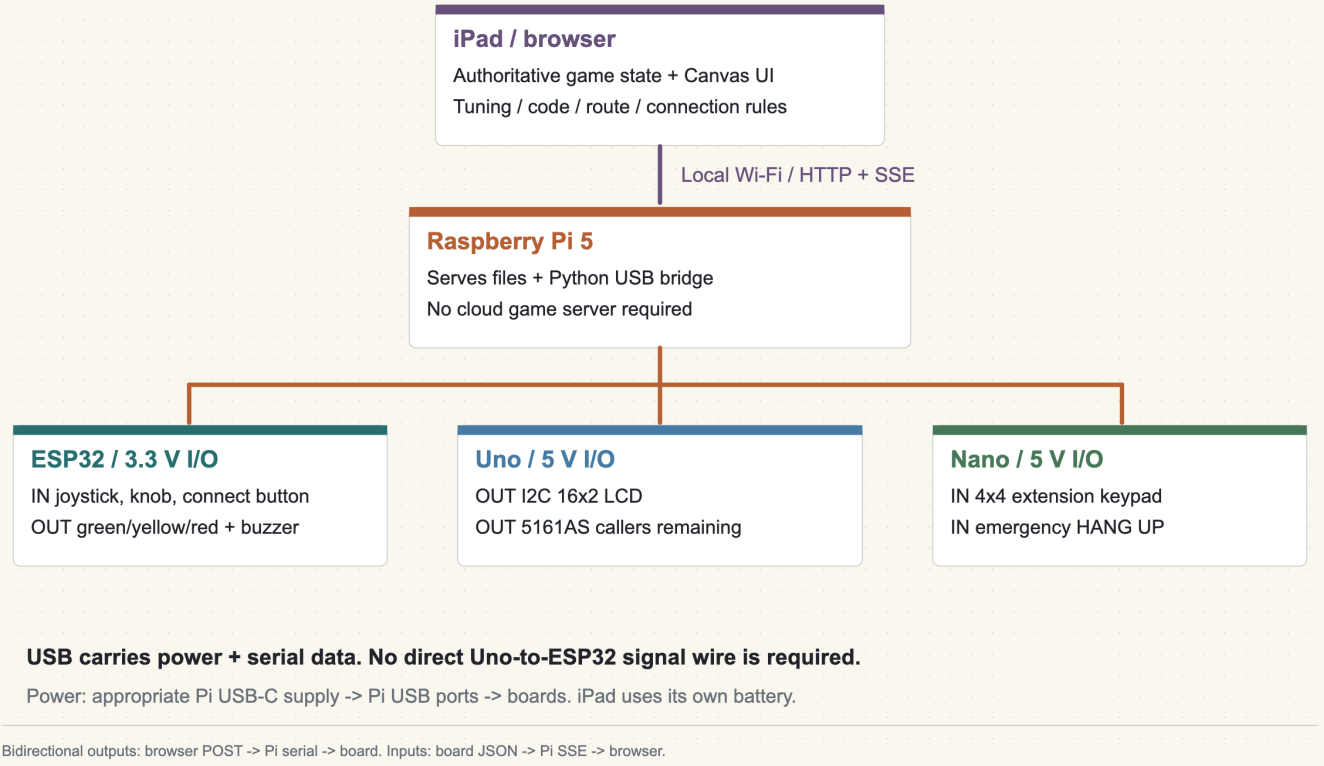
An ordinary telephone exchange becomes an impossible service desk. Demolished rooms, an attentive operator, a paper call log and illuminated destination pumpkins give the electronics a narrative role. Tension comes from reading the desk while the eye opens.

The carrier scale is fictional ADC-driven tuning, not RF reception. Callers are text, not recorded human voices. The original six arcade cartridges remain available.

Where the game actually runs

01 / THE SIGNAL PATH

One browser game, one Linux host, three independent USB peripherals.



USB carries power + serial data. No direct Uno-to-ESP32 signal wire is required.

Power: appropriate Pi USB-C supply -> Pi USB ports -> boards. iPad uses its own battery.

Bidirectional outputs: browser POST -> Pi serial -> board. Inputs: board JSON -> Pi SSE -> browser.

The iPad browser owns the game rules and renders the scene. The Pi hosts files and translates between HTTP/live browser events and three independent USB serial ports. The boards sample inputs or drive outputs; they do not independently decide game outcomes.

One host, three stations

ESP32: analog controls, main button, indicators and buzzer.
Uno: LCD plus seven-segment output.
Nano: matrix keypad plus emergency hang-up.

Local operation

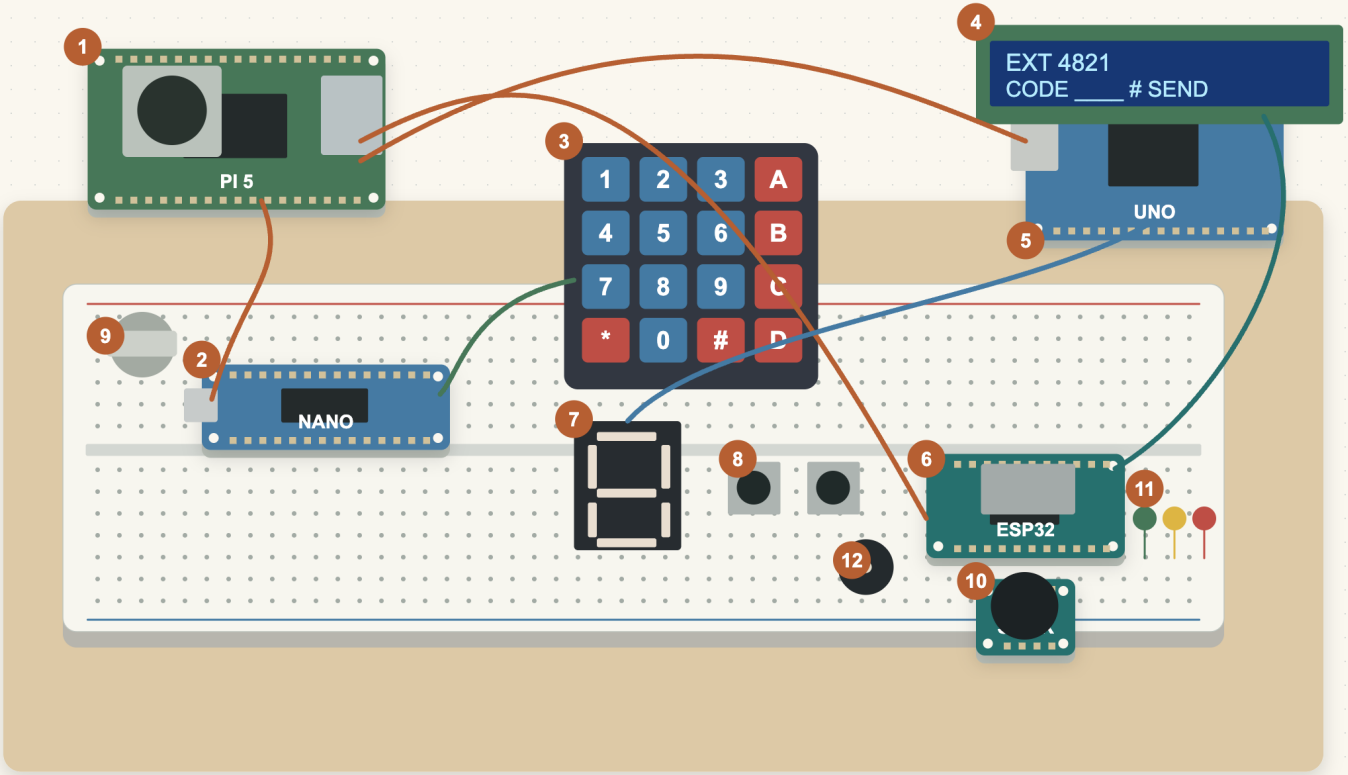
The Pi and browser need a reachable local network. No cloud game service or Mac is required after Pi setup. A Mac forwarding gateway was used during development, but it is optional for an iPad connected directly to the Pi.

Three microcontrollers are a modular reuse choice, not a minimum requirement. Separate stations simplify individual bring-up and add USB/cabling complexity.

The photographed console, clarified

02 / PHOTO-BASED ASSEMBLY MAP

A clean 2.5D reconstruction of the photographed prototype; geometry and cable paths are normalized.



01 Pi 5	Host + USB power	**02 Nano**	Keypad station	**03 Keypad**	Extension entry	**04 LCD**	Private extension
05 Uno	Display station	**06 ESP32**	Main controls	**07 5161AS**	6 callers -> 0	**08 Buttons**	Connect / hang up
09 Knob	Carrier tuning	**10 Joystick**	Route selection	**11 LEDs**	Safe / warn / release	**12 Buzzer**	Call / warning cues

Spatial guide, not a scale drawing or circuit simulation. Photos 3265-3269 establish the physical component set.

The parts on the desk

Qty	Part / implementation
1 each	Pi 5, ESP32 development board, Uno-compatible board, classic Nano-compatible board; iPad or browser display.
1 each	Analog joystick; potentiometer (value unconfirmed); 4x4 membrane keypad.
2	External momentary buttons: connect and hang-up. Joystick also has its own click switch.
1 each	16x2 I2C LCD backpack at 0x27; 5161AS common-cathode digit.
3	Green, yellow and red indicator LEDs.
10	Nominal 330-ohm resistors: 3 LED branches + 7 individual segment branches.
1 each	Two-pin buzzer (exact type/rating unconfirmed); nominal 1-kilohm series resistor.
3 + wiring	USB data cables; breadboards, jumpers and suitable Pi USB-C supply.

Keep the voltage domains explicit

Power the Pi through its suitable USB-C input. The three USB ports provide the boards with power and serial data. Use ESP32 3.3 V for the analog control circuits; use Uno 5 V for its LCD. Each circuit returns to its own board ground. There is no required direct Uno/Nano logic wire to ESP32 input pins.

Resistor identification

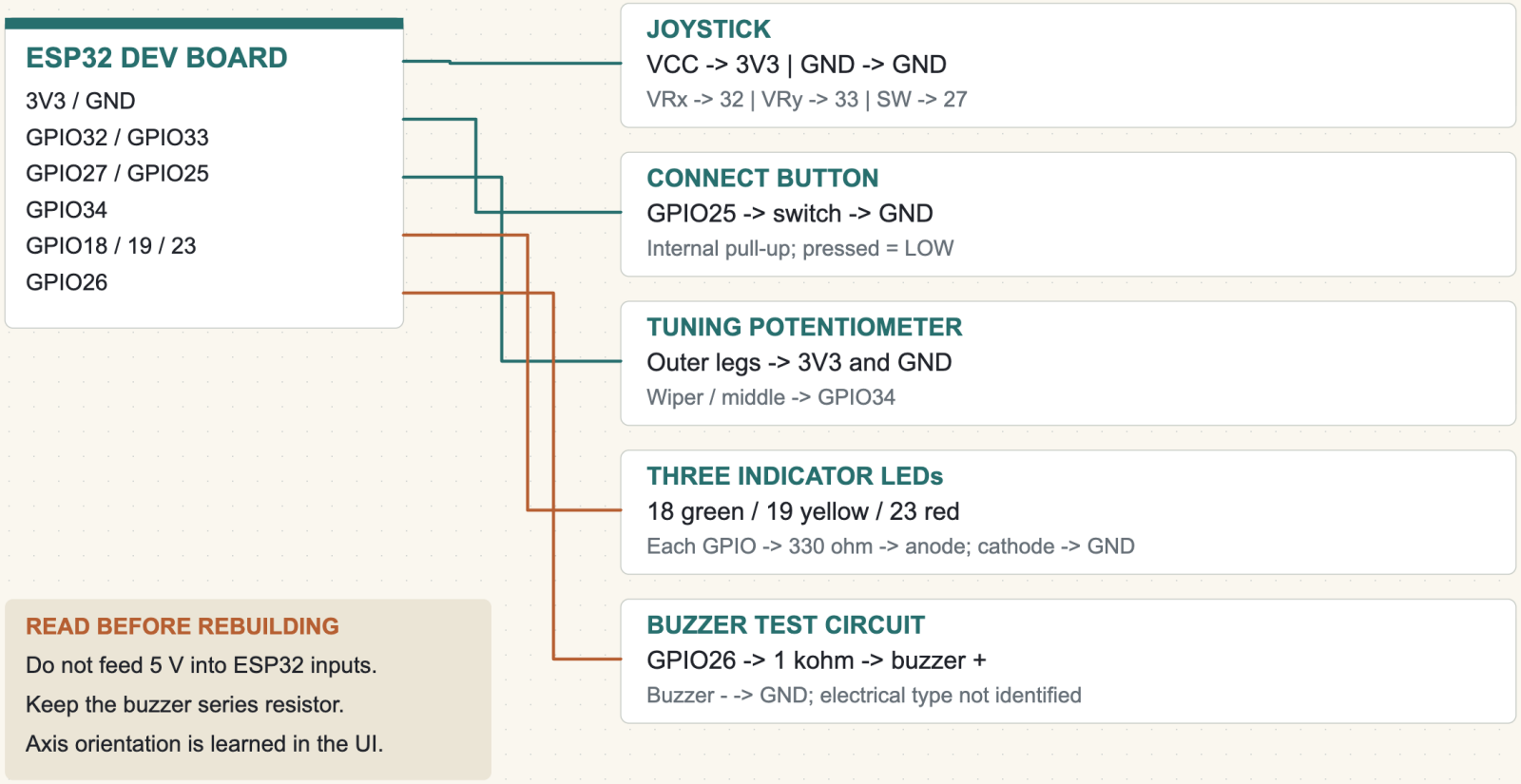
330 ohms, five bands: orange / orange / black / black / brown.
1 kilohm, five bands: brown / black / black / brown / brown.
These are nominal band readings; no resistance measurement is claimed.

Buzzer limitation: the current-limited GPIO26 branch records this prototype, not a general driver design. Identify the device and its current/voltage needs before reproducing it with another buzzer. Supply model, total current, potentiometer value and costs are not established.

ESP32 - controls and indicators

03 / ESP32 CONTROL STATION

3.3 V signals. GPIO numbers are board labels, not physical header positions.



Firmware netlist is authoritative. Lead colours/hidden breadboard rows cannot be established from photos alone.

Uno - display station

04 / UNO DISPLAY STATION

LCD and seven-segment share one Uno, with separate responsibilities.

ARDUINO UNO

USB from Pi

LCD: 5V / GND / A4 / A5

Segments: digital 2 through 8

16x2 I2C LCD @ 0x27

GND -> Uno GND

VCC -> Uno 5V

SDA -> A4 | SCL -> A5

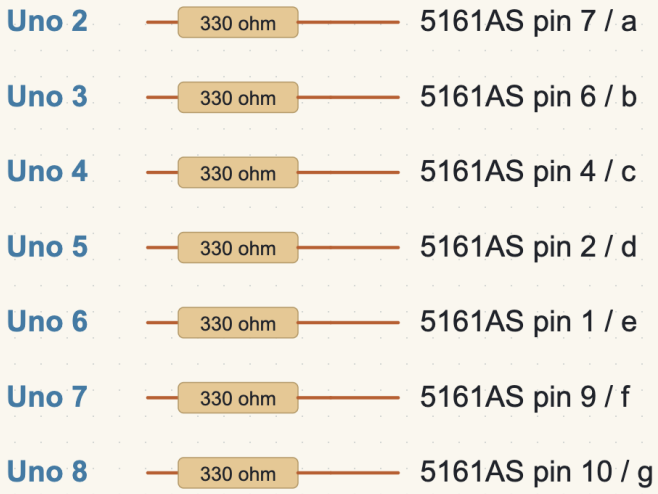
NOT THE NANO

The display driver runs on the Uno.

Nano D2-D9 belong to the keypad.

The photographed build exposed this easy-to-make board-name mistake.

ONE 330-OHM RESISTOR PER SEGMENT



COMMON CATHODE / FRONT VIEW

Pins 3 and 8 -> Uno GND. Pin 5 (decimal) unused.

Top, left to right: 10 9 8 7 6

Bottom, left to right: 1 2 3 4 5 | dot at bottom right

Each segment needs its own resistor. Do not replace all seven resistors with one common-ground resistor.

Nano - keypad station

05 / NANO KEYPAD STATION

4x4 passive matrix + one active-low HANG UP button.

NANO / ATmega328P

USB serial to Pi

No direct connection to ESP32 I/O

D10 button uses INPUT_PULLUP

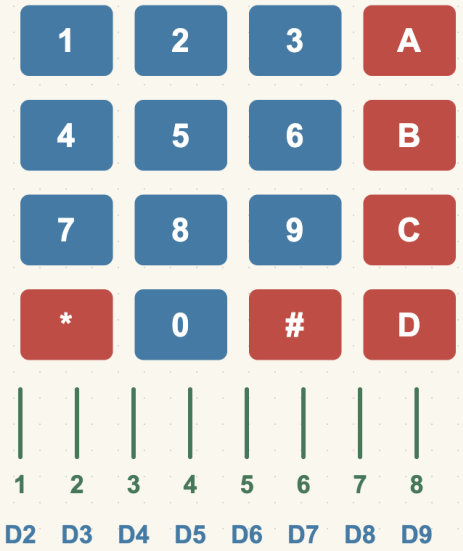
BUTTON

Nano D10 -> switch -> Nano GND

No external pull-up required.

No 5V/GND wire on the keypad tail.

KEYPAD FRONT VIEW / CABLE POINTING DOWN



Pin 1 is the leftmost cable contact in this front-view orientation.

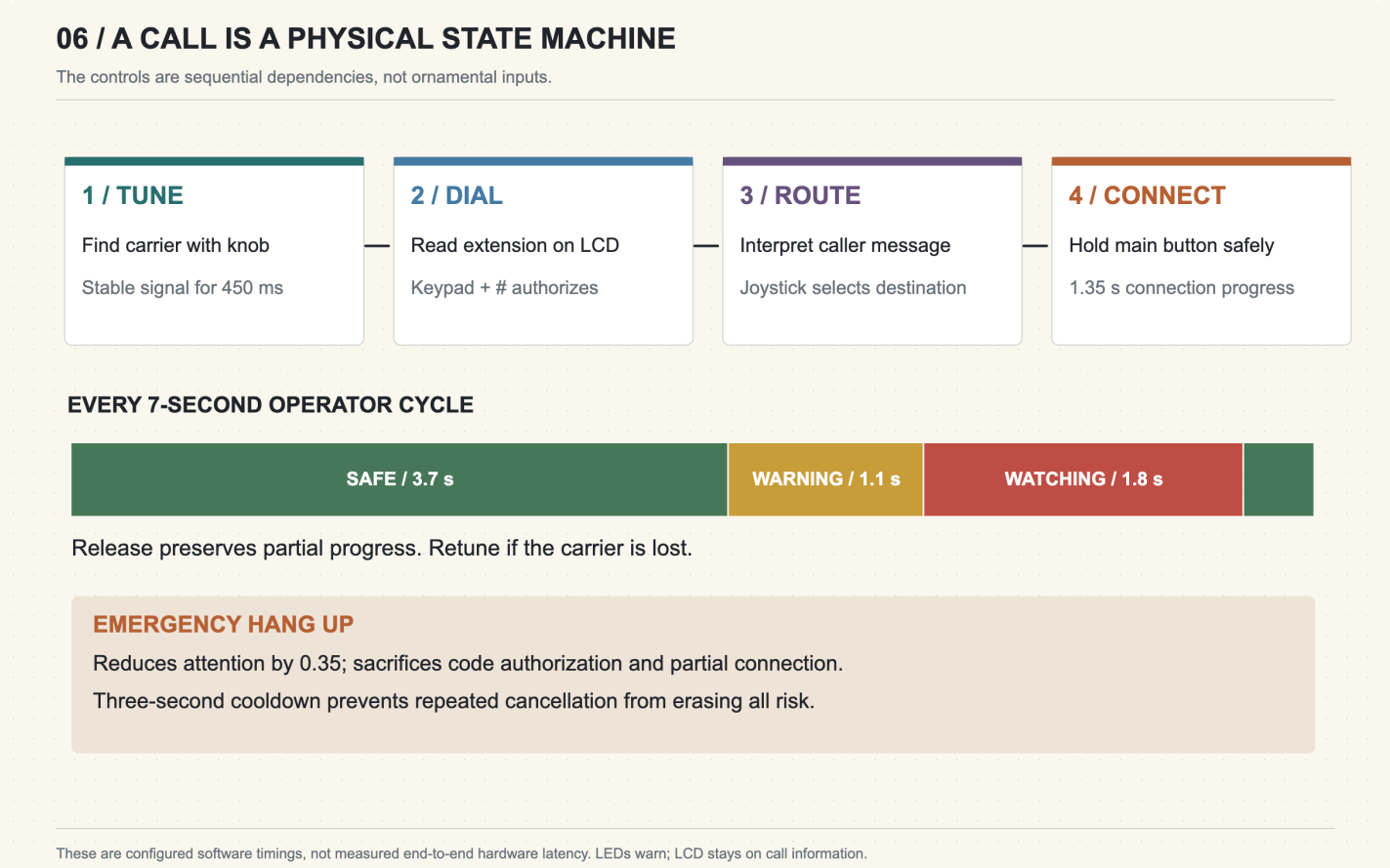
Physical connector contacts 1-8 -> Nano D2-D9.

Firmware interpretation for this specimen:

Rows = D9,D8,D7,D6; columns = D5,D4,D3,D2. Keep the physical D2-D9 wiring order above.

A reversed tail initially decoded 1,2,3,4 as D,C,B,#. The scan map was corrected in firmware; do not reverse it again.

A winnable night shift



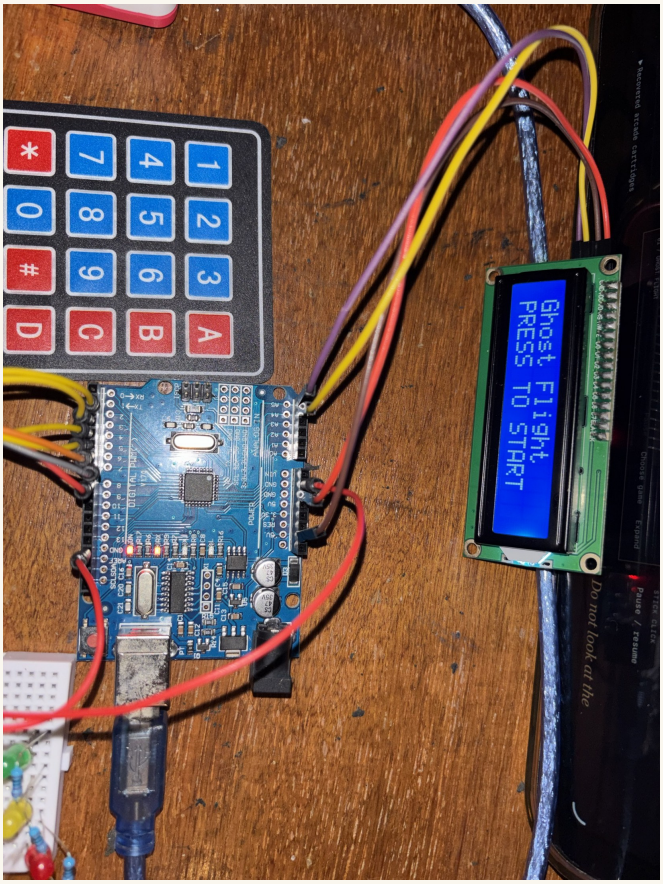
A safe interval is long enough

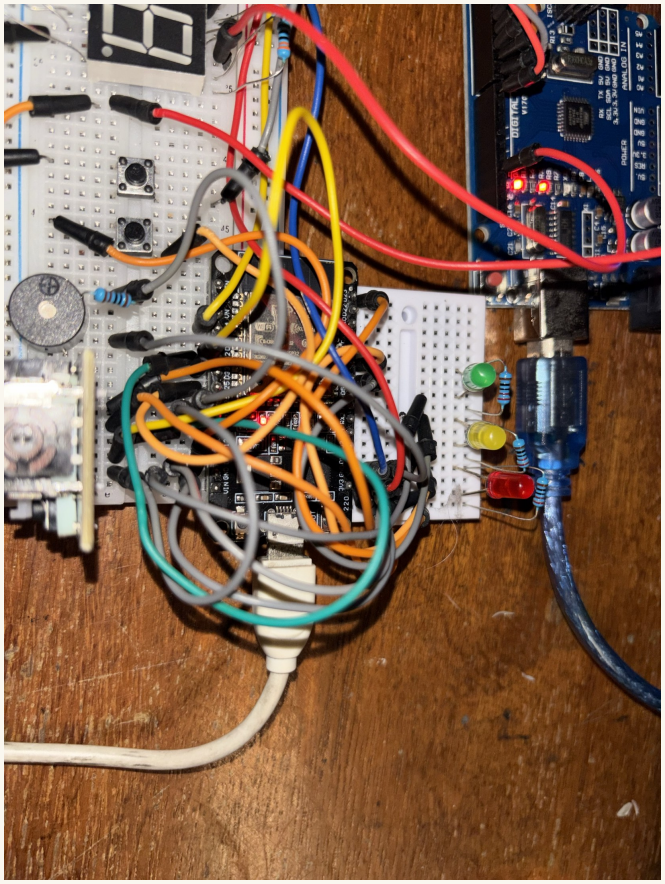
A correct connection requires 1.35 seconds of progress. The initial safe interval is 3.7 seconds, followed by a 1.1-second warning and 1.8 seconds watched. The final 0.4 seconds are safe again. Releasing pauses progress, so a careful player can split a connection across cycles.

Action	Configured effect
Hold while watched	Attention rises 0.65 per second.
Hold on wrong destination	Attention rises 0.20 per second.
Submit wrong extension	Attention increases by 0.15.
Release / remain quiet	Attention falls by 0.12 per second.
Hang up	Attention -0.35; clears current work and changes extension; 3-second cooldown.

Values are configured game constants, not measured hardware latency. The eye and LEDs provide timing cues; the LCD remains on the call. Browser rehearsal has reached the six-call ending.

Control and display details





Left: Uno and I2C display. The visible text is the Ghost Flight title/menu, not Please Hold call data. Right: ESP32 controls, buzzer and LED branches. Photos preserve the photographed prototype, including loose jumpers.

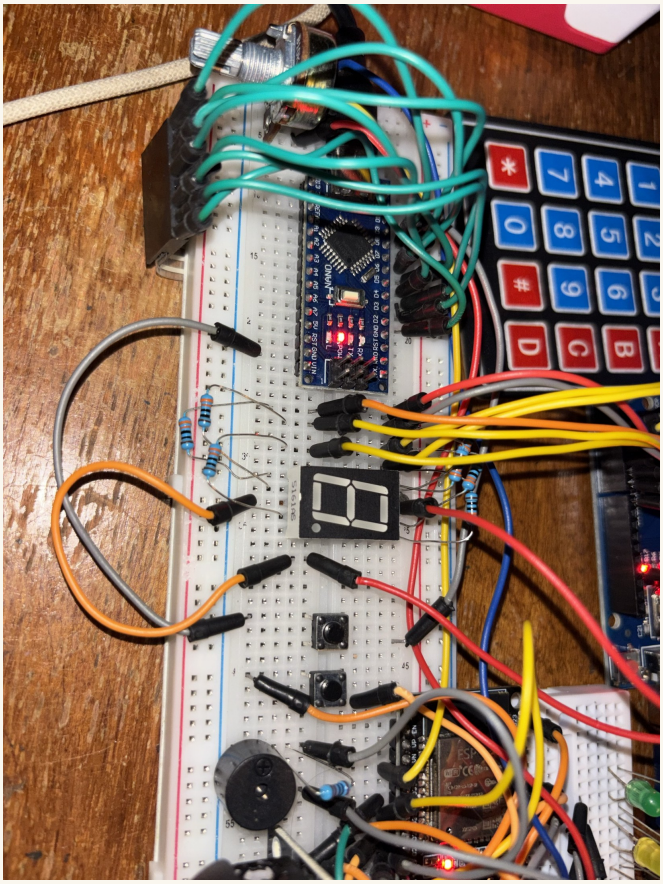
What a lit backlight proves

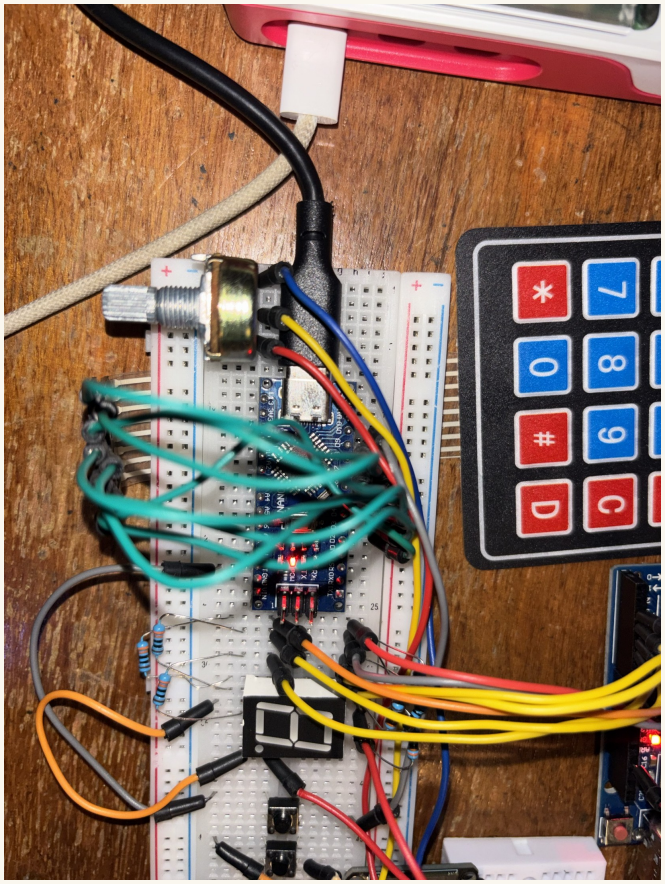
A backlight establishes that at least part of the module is powered. It does not prove that SDA/SCL are connected, that the address is correct or that initialization succeeded. Blocks and random characters should trigger a wiring/contrast check and a fresh initialization.

What the photographs cannot prove

They establish the physical component set and arrangement. Some wires disappear under other wires, and breadboard rail breaks are not all visible. The logical atlas and netlist therefore follow the current firmware pin assignments rather than invented row coordinates.

The extra board earns its place





Nano, keypad, tuning knob and segment details in the builder photographs. The segment occupies the same breadboard area as the Nano, but its seven output wires belong to the Uno.

Keypad is an authorization device

Entering the extension and selecting the destination solve different problems. The Nano gives the player a real typing task while the joystick remains a routing control. Its separate button is a costly escape action, not a second-player controller.

Connector order matters

Read the keypad drawing from the front with the tail down. The observed reverse decoding was corrected in the firmware row/column arrays. Test all sixteen symbols on the actual specimen; keypad membrane pinouts can vary.

Commission it in layers

1. Verify unpowered wiring

Compare every connection with the atlas and CSV. Check ground, common-cathode pins and split breadboard rails. Keep a separate resistor on each LED/segment. Confirm the buzzer type before copying its drive branch.

2. Flash matching sketches

Use haunted_controller for ESP32, lcd_station for Uno and keypad_station for Nano. Install the relevant Arduino cores and hd44780 1.3.2. Release serial ports before upload; the Nano specimen used the current ATmega328P bootloader.

3. Identify the USB stations

Connect all three data cables. Identify each board using persistent /dev/serial/by-id paths on Linux. Avoid relying on ttyUSB enumeration order. The runtime serial speed is 115200 baud.

4. Start the host

Run Python 3 server.py with --serial, --lcd-serial and --keypad-serial pointing to your own devices. The server uses Python standard-library modules. See BUILD-GUIDE.md for the portable command and toolchain examples.

5. Open and test

On the same network open http://PI_LAN_ADDRESS:8765/. Check /api/controller and /api/lcd, then observe every physical output. Test the full keypad and run browser direction learning on the iPad.

6. Play and inspect failure behavior

Run Please Hold with the required stations online. A required-device loss pauses hardware play. Verify release, detuning, wrong code, hang-up and a successful decrement before attempting a full shift.

Browser events use SSE; output HTTP requests are translated to serial commands. LCD framing is D + 16 characters + | + 16 characters + newline. S0...S9 sets the digit; S- blanks; T starts the segment test. Acknowledgements describe firmware state, not optical visibility.

The failures worth keeping

LCD: two different failure layers

Loose connections produced an I2C address NACK. After the address acknowledged, repeated initialization still returned -4. The hd44780 automatic instance search advanced across retries; constructing hd44780_I2Cexp lcd(0x27) avoided that repeated auto-location path. The final report returned I2C 0 and init 0. Separate telemetry prevented a USB connection from masquerading as display health.

Keypad: a mirrored connector

The first observed 1,2,3,4 presses arrived as D,C,B,#. Firmware rows D9,D8,D7,D6 and columns D5,D4,D3,D2 corrected this specimen. Reversing the physical cable again would undo that correction.

Seven-segment: correct wires, wrong board

The display was initially wired to the Nano while the driver lived on the Uno. Completing the ground and moving the outputs to the proper board resolved the digit test. The builder subsequently confirmed the physical display worked.

USB: partial survival hid the fault

The Pi kernel logged an xHCI host-controller failure. ESP32 and Uno disappeared while the Nano remained attached. Reboot restored all three serial devices. The root cause is unresolved; a non-throttled status snapshot does not establish the absence of a power problem.

Joystick: orientation is a mapping problem

Physical up/down initially moved the character left/right. Browser direction learning maps the installed module orientation to the game axes and persists it per browser origin. The iPad must learn its own mapping.

Evidence without inflated claims

Evidence	What it establishes
30 automated tests passed	Engine/display behavior checked with Node; log included in validation/node-tests.txt.
Six-call browser rehearsal	The rule loop can reach its ending using simulated controls. Not proof of a physical six-call win.
Pi output integration	Simulated inputs drove real Pi-connected outputs: LCD extension acknowledgement and digit6 to5. Not optical proof of every display state.
Physical confirmations	Builder reported working LED, knob and digit behavior during commissioning. Complete corrected-keypad matrix verification was not retained.
Five photographs	Actual components and build arrangement. LCD image is a Ghost Flight menu.
Gameplay recording reported	User reports a recording exists. Media file/link has not been provided or reviewed.

What to verify next

Review the continuous gameplay recording; confirm every keypad symbol; secure LCD connectors; identify the buzzer; measure current and response latency; perform a sustained physical six-call playthrough. These are explicit follow-ups, not completed measurements.

Configured rates are not benchmarks

ESP32 sampling/reporting is configured for 20 ms with 15 ms button debounce. The knob averages eight ADC samples and applies a 0.25 exponential filter. Nano debounce is 25 ms with a 100 ms heartbeat. Browser output scheduling and Wi-Fi/USB add delay; no measured input-to-light latency is claimed.

Physical-controller loss and browser backgrounding pause a run. Rehearsal offers a development route through the same console mechanics. See EVIDENCE.md for the full claim ledger.

A build others can inspect

Inside the companion package

File / directory	Use
FORUM-POST.md	First-person project article; add reviewed gameplay media before publication.
diagrams/*.svg and *.png	Editable vector atlas plus forum-ready raster exports.
wiring-netlist.csv / .json	Portable connection reference for all three boards.
BUILD-GUIDE.md	Toolchain, host command, commissioning and source map.
EVIDENCE.md / validation/	Evidence boundaries and the automated test transcript.
Dead-Channel-Source.zip	Curated runtime, firmware and tests; excludes machine caches, binaries and deployment scripts.

Credits and illustration status

Physical build and supplied photographs: Adriel Magalona. Game software, documentation and Canvas artwork were developed with AI coding assistance. The Halloween illustration was AI-generated; its prompt is included in web/art/HALLOWEEN-PROMPT.md. Procedural browser sound is generated in code. Arduino/ESP32 toolchains and Bill Perry's hd44780 library retain their own attribution.

The assembly map is a normalized 2.5D reconstruction. No Tinkercad simulation, measured electrical certification or original hand-drawn illustration is claimed. No public repository or gameplay URL has yet been supplied. This dossier is a local draft for review, not an externally published entry.

Research and technical references

element14: 2025 winners

element14: 2025 project roundup

element14: 2026 competition brief

5161AS common-cathode reference

Access/review edition: 30 October 2026. Detailed winner comparisons and direct entry links are in research/2025-BENCHMARK.md. Judging outcomes cannot be promised. Review current rules and complete terms before submission.